

TD1 — Programmation OpenMP

Parallélisme avec OpenMP (2025–2026)

Auteur : Yahdhih Abdel Wedoud

Date : 22 janvier 2026

Résumé. Ce rapport répond aux questions **Q1 à Q9** du TD1 sur OpenMP. Il couvre le passage de OpenMP à Pthread, la parallélisation de boucles, les clauses de réduction, et l'implémentation d'un produit matrice creuse–vecteur parallèle.

Table des matières

1	Contexte et objectifs	1
2	Partie I : Passage de OpenMP à Pthread	2
2.1	Analyse du programme OpenMP initial	2
2.2	Parallélisation de la boucle <code>for</code>	4
2.3	Recherche parallèle du maximum	5
3	Partie II : Produit matrice creuse–vecteur	6
3.1	Structure de données	6
3.2	Benchmarks	8
4	Synthèse des directives OpenMP utilisées	8
5	Conclusion	8

1 Contexte et objectifs

L'objectif de ce TD est de se familiariser avec l'API **OpenMP** pour la programmation parallèle en mémoire partagée. Les points abordés sont :

- Compréhension du modèle d'exécution OpenMP (fork-join);
- Passage d'un programme OpenMP vers Pthread et inversement;
- Parallélisation de boucles avec `#pragma omp for`;
- Synchronisation avec `critical`, `master`;
- Utilisation de la clause `reduction`;
- Application au produit matrice creuse–vecteur.

Compilation type (macOS avec GCC) :

```
gcc-15 -fopenmp -Wall -O2 fichier.c -o programme
```

Exécution avec N threads :

```
OMP_NUM_THREADS=4 ./programme <args>
```

2 Partie I : Passage de OpenMP à Pthread

2.1 Analyse du programme OpenMP initial

Le fichier `prog_omp.c` contient le code suivant :

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char **argv)
5  {
6      int N = atoi(argv[1]);
7      int sum = 0;
8
9      #pragma omp parallel shared(sum,N)
10     {
11         int i, sum_loc = 0;
12
13         #pragma omp master
14         {
15             printf("Je suis le maitre !\n");
16         }
17
18         for(i = 0 ; i < N ; i++)
19         {
20             sum_loc += i;
21         }
22
23         #pragma omp critical
24         {
25             sum += sum_loc;
26         }
27     }
28
29     printf("sum = %d\n", sum);
30     return 0;
31 }
```

Exécution avec 4 threads et N = 1000 :

```
$ OMP_NUM_THREADS=4 ./prog_omp 1000
Je suis le maitre !
sum = 1998000
```

Explication. La boucle `for` n'est pas précédée de `#pragma omp for`, donc chaque thread exécute toute la boucle de 0 à N-1.

Calcul :

- Somme pour un thread : $\sum_{i=0}^{999} i = \frac{999 \times 1000}{2} = 499500$
- Avec 4 threads : $4 \times 499500 = 1998000$

Conclusion. La valeur de `sum` dépend du nombre de threads car la boucle n'est pas distribuée.

Objectif. Reproduire le comportement exact du programme OpenMP avec l'API Pthread.

```
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef struct {
6      int thread_id;
7      int N;
8  } thread_arg_t;
9
10 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
11 int sum = 0;
12
13 void *sum_thread(void *arg)
14 {
15     thread_arg_t *p = (thread_arg_t *)arg;
16     int sum_loc = 0;
17
18     /* Equivalent de #pragma omp master */
19     if (p->thread_id == 0) {
20         printf("Je suis le maitre !\n");
21     }
22
23     /* Chaque thread fait toute la boucle */
24     for (int i = 0; i < p->N; i++) {
25         sum_loc += i;
26     }
27
28     /* Equivalent de #pragma omp critical */
29     pthread_mutex_lock(&mutex);
30     sum += sum_loc;
31     pthread_mutex_unlock(&mutex);
32
33     free(p);
34     return NULL;
35 }
36
37 int main(int argc, char **argv)
38 {
39     if (argc < 3) {
40         fprintf(stderr, "Usage: %s <nthreads> <N>\n", argv[0]);
41         return 1;
42     }
43
44     int nthreads = atoi(argv[1]);
45     int N = atoi(argv[2]);
46     pthread_t tids[nthreads];
47
48     for (int i = 0; i < nthreads; i++) {
49         thread_arg_t *p = malloc(sizeof(thread_arg_t));
50         p->thread_id = i;
```

```

51     p->N = N;
52     pthread_create(&tids[i], NULL, sum_thread, p);
53 }
54
55 for (int i = 0; i < nthreads; i++) {
56     pthread_join(tids[i], NULL);
57 }
58
59 printf("sum = %d\n", sum);
60 return 0;
61 }

```

Correspondances OpenMP ↔ Pthread :

OpenMP	Pthread
#pragma omp parallel	pthread_create (N fois)
#pragma omp master	if (thread_id == 0)
#pragma omp critical	pthread_mutex_lock/unlock
Fin de région parallèle	pthread_join

2.2 Parallélisation de la boucle for

Problème. Dans le code initial, la boucle `for` est exécutée entièrement par chaque thread.

Solution. Ajouter `#pragma omp for` pour **distribuer** les itérations :

```

1  #pragma omp parallel shared(sum, N)
2  {
3      int sum_loc = 0;
4
5      #pragma omp master
6      {
7          printf("Je suis le maitre !\n");
8      }
9
10     /* La boucle est maintenant distribuee */
11     #pragma omp for
12     for (int i = 0; i < N; i++)
13     {
14         sum_loc += i;
15     }
16
17     #pragma omp critical
18     {
19         sum += sum_loc;
20     }
21 }

```

Résultat avec 4 threads et $N = 1000$:

```

$ OMP_NUM_THREADS=4 ./prog_omp_for 1000
Je suis le maitre ! (thread 0 sur 4)
sum = 499500
Valeur attendue: 499500

```

Explication. Avec `#pragma omp for`, chaque thread traite un sous-ensemble des itérations (par défaut, découpage statique en blocs contigus). Le résultat est maintenant **indépendant** du nombre de threads.

Question. Quelle directive assure que plusieurs threads ne modifient pas `sum` simultanément ?

Toggle : Réponse. La directive `#pragma omp critical` définit une **section critique** : un seul thread à la fois peut exécuter le bloc associé.

Fonctionnement :

- Équivalent à un `pthread_mutex_lock/unlock` implicite ;
- Garantit l'exclusion mutuelle ;
- Peut être nommée : `#pragma omp critical(nom)` pour distinguer plusieurs sections.

Alternative. La directive `#pragma omp atomic` pour des opérations simples (incréméntation, etc.) est plus efficace qu'un `critical`.

Question. OpenMP propose-t-il un autre moyen plus simple pour calculer `sum` ?

Réponse. Oui, la clause `reduction` :

```
1 int sum = 0;
2
3 #pragma omp parallel for reduction(+:sum)
4 for (int i = 0; i < N; i++)
5 {
6     sum += i;
7 }
8
9 printf("sum = %d\n", sum);
```

Fonctionnement de `reduction(+:sum)` :

1. Chaque thread reçoit une copie locale de `sum` initialisée à 0 (élément neutre de +) ;
2. Chaque thread accumule dans sa copie locale ;
3. À la fin de la région parallèle, toutes les copies sont combinées avec l'opérateur +.

Avantages par rapport à `critical` :

- **Plus simple** : pas de variable locale explicite ni de section critique ;
- **Plus efficace** : pas de verrou à chaque itération, combinaison finale optimisée ;
- **Moins de risque d'erreur** : gestion automatique par le compilateur.

Opérateurs supportés : +, -, *, &, |, ^, &&, ||, max, min.

2.3 Recherche parallèle du maximum

Objectif. Reprendre le calcul parallèle du maximum d'un tableau en OpenMP.

Méthode 1 : Clause `reduction(max:)` (la plus simple)

```
1 int max_par_reduction(int *tab, int nelt)
2 {
3     int maxv = 0;
4
5     #pragma omp parallel for reduction(max:maxv)
```

```

6   for (int i = 0; i < nelt; i++)
7   {
8       if (tab[i] > maxv)
9           maxv = tab[i];
10  }
11
12  return maxv;
13 }

```

Méthode 2 : Max local + critical (équivalent Pthread)

```

1  int max_par_critical(int *tab, int nelt)
2  {
3      int maxv_global = 0;
4
5      #pragma omp parallel shared(maxv_global)
6      {
7          int maxv_local = 0;
8
9          #pragma omp for
10         for (int i = 0; i < nelt; i++)
11         {
12             if (tab[i] > maxv_local)
13                 maxv_local = tab[i];
14         }
15
16         #pragma omp critical
17         {
18             if (maxv_local > maxv_global)
19                 maxv_global = maxv_local;
20         }
21     }
22
23     return maxv_global;
24 }

```

Test de validation :

```

$ OMP_NUM_THREADS=4 ./max_tab_omp 10000
=== Resultats ===
Max sequentiel:          1000
Max parallel (reduction): 1000
Max parallel (critical): 1000

PASSED: Toutes les methodes donnent le meme resultat

```

3 Partie II : Produit matrice creuse–vecteur

3.1 Structure de données

La matrice creuse est stockée au format **CSR simplifié** (Compressed Sparse Row) :

- `ncol[i]` : nombre d'éléments non nuls sur la ligne i ;
- `col[i][k]` : colonne du k -ième élément non nul de la ligne i ;
- `elt[i][k]` : valeur du k -ième élément non nul de la ligne i .

Le produit $Y = A \times X$ se calcule par :

$$Y_i = \sum_{k=0}^{\text{ncol}[i]-1} A_{i,\text{col}[i][k]} \times X_{\text{col}[i][k]}$$

Analyse. Chaque ligne i du résultat Y est indépendante des autres : pas de dépendances entre $Y[i]$ et $Y[j]$ pour $i \neq j$.

Stratégie. Paralléliser la boucle externe sur les lignes avec `#pragma omp parallel for`.

```

1 void prod_mat_vec_par(sparse_matrix_t *A, vector_t *X, vector_t *Y)
2 {
3     assert(A->N == X->N);
4     assert(Y->N == X->N);
5
6     int i;
7
8     #pragma omp parallel for private(i)
9     for (i = 0; i < A->N; i++)
10    {
11        double accu = 0.0;
12
13        for (int k = 0; k < A->ncol[i]; k++)
14        {
15            int j = A->col[i][k];
16            accu += A->elt[i][k] * X->elt[j];
17        }
18
19        Y->elt[i] = accu;
20    }
21 }
```

Points clés :

- `private(i)` : chaque thread a sa propre copie de l'indice de boucle ;
- `accu, k, j` : variables locales au bloc, donc automatiquement privées ;
- Pas besoin de `critical` car chaque thread écrit dans une case différente de Y .

Objectif. Vérifier si deux vecteurs sont égaux en parallèle.

```

1 int is_equal(vector_t *vec1, vector_t *vec2)
2 {
3     int i, nb_diff;
4
5     assert(vec1->N == vec2->N);
6
7     nb_diff = 0;
8
9     #pragma omp parallel for reduction(+:nb_diff)
10    for (i = 0; i < vec1->N; i++)
11    {
12        if (vec1->elt[i] != vec2->elt[i])
13        {
14            nb_diff++;
15        }
16    }
```

```

15         }
16     }
17
18     return nb_diff == 0;
19 }

```

Explication. On utilise `reduction(+:nb_diff)` pour compter le nombre de différences. Chaque thread compte localement, puis les compteurs sont sommés à la fin.

3.2 Benchmarks

Paramètres : $N = 100000$ éléments, `niter` = 100 itérations.

Threads	Temps (s)	Speedup	Efficacité
1	0.067	1.00	100%
2	0.036	1.86	93%
4	0.026	2.58	64%
8	0.022	3.05	38%

Analyse :

- **Speedup** = $\frac{T_1}{T_p}$ où T_1 est le temps séquentiel et T_p le temps avec p threads ;
- **Efficacité** = $\frac{\text{Speedup}}{p} \times 100\%$;
- L'efficacité diminue avec le nombre de threads à cause :
 - de l'overhead de création/synchronisation des threads ;
 - du déséquilibre de charge (lignes de longueurs différentes) ;
 - des effets de cache (faux partage, contention mémoire).

Remarque. Le speedup de 3x avec 8 threads est typique pour ce type de calcul irrégulier sur matrice creuse.

4 Synthèse des directives OpenMP utilisées

Directive/Clause	Description
<code>#pragma omp parallel</code>	Crée une équipe de threads (région parallèle)
<code>#pragma omp for</code>	Distribue les itérations d'une boucle entre les threads
<code>#pragma omp parallel for</code>	Combinaison des deux précédentes
<code>#pragma omp master</code>	Exécuté uniquement par le thread maître (ID 0)
<code>#pragma omp critical</code>	Section critique (exclusion mutuelle)
<code>shared(var)</code>	Variable partagée entre tous les threads
<code>private(var)</code>	Chaque thread a sa propre copie
<code>reduction(op:var)</code>	Réduction avec l'opérateur <code>op</code> sur <code>var</code>

5 Conclusion

Ce TD a permis de :

1. Comprendre le modèle d'exécution OpenMP et ses équivalences avec Pthread ;
2. Maîtriser la parallélisation de boucles avec `#pragma omp for` ;

3. Utiliser efficacement la clause **reduction** pour les opérations de réduction ;
4. Appliquer ces concepts à un problème concret (produit matrice creuse–vecteur) ;
5. Mesurer et analyser les performances obtenues.

Points importants à retenir :

- Sans **#pragma omp for**, chaque thread exécute **toute** la boucle ;
- **reduction** est plus simple et souvent plus efficace que **critical** ;
- Le speedup réel dépend de la nature du problème (régularité, granularité) ;
- OpenMP simplifie grandement la parallélisation par rapport à Pthread.